Búsqueda lineal: examinarlo todo, en orden

Adaptación a C#

Introducción

Tienes una lista de nombres — digamos, la lista de invitados a una boda — y necesitas saber si alguien en particular está en ella. No hay ningún orden en la lista; alguien fue escribiendo los nombres a medida que la gente confirmaba su asistencia. ¿Qué haces? Lees la lista, de arriba abajo, hasta que encuentras el nombre o se acaba la lista. Eso es todo. Ese es el algoritmo. Y nada puede hacerlo mejor cuando la entrada no tiene estructura. La búsqueda lineal es el único algoritmo para búsqueda no estructurada, salvo simetrías.

El *adversario* — alguien que puede reordenar la lista mientras no miras, específicamente para hacerte revisar cada elemento antes de que puedas responder — volverá a aparecer varias veces. La búsqueda lineal es donde lo conoces.

1. Una secuencia y nada más

El escenario es el más simple posible. Tienes una secuencia de elementos — una colección ordenada finita donde cada elemento tiene un índice y puedes pedir el elemento i-ésimo. No sabes nada más. Los elementos podrían estar ordenados, o no. Podría haber duplicados, o no. Simplemente recorres los índices.

En C#, la forma más genérica de representar una secuencia indexable es `IReadOnlyList<T>`. Expone `Count` e indexación, que es exactamente lo que necesitamos: nada más, nada menos.

```
public static bool Find<T>(T x, IReadOnlyList<T> items)
{
    for (int i = 0; i < items.Count; i++)
    {
        if (EqualityComparer<T>.Default.Equals(x, items[i]))
            return true;
    }
    return false;
}
```

Listing 1: Búsqueda lineal: `Find`

Todo el algoritmo en cinco líneas efectivas. Recorres la secuencia; si en algún momento hay una coincidencia, te detienes y respondes que sí; si llegas al final, respondes que no. El retorno temprano es lo que hace que el caso promedio sea mejor que el caso peor, aunque solo por un factor constante.

Ejecutándolo con algunas entradas:

```
Console.WriteLine(Linear.Find(3, new[] { 1, 2, 3, 4, 5 })); // True  (posicion 2, 3 elementos)
Console.WriteLine(Linear.Find(9, new[] { 1, 2, 3, 4, 5 })); // False (recorrio los 5)
Console.WriteLine(Linear.Find("apple", new[] { "banana", "cherry", "apple" })); // True
Console.WriteLine(Linear.Find(1, Array.Empty<int>()));     // False (vacio)
```

```
True
False
True
False
```

2. De “¿está ahí?” a “¿dónde?” y “¿cuántos?”

`Find` te dice si un elemento existe. A menudo quieres más. ¿Dónde está? O, cuando los duplicados importan, ¿cuántos hay?

La primera variación es `Index` — el menor índice i donde `items[i] == x`. Si el elemento no está presente, devolvemos `null` (representado como `int?`) en lugar de lanzar una excepción, para que quien llama no tenga que envolver la llamada en un `try`.

```
public static int? Index<T>(T x, IReadOnlyList<T> items)
{
    for (int i = 0; i < items.Count; i++)
    {
        if (EqualityComparer<T>.Default.Equals(x, items[i]))
            return i;
    }
    return null;
}
```

Listing 2: `Index`: primera ocurrencia

```
Console.WriteLine(Linear.Index(3, new[] { 1, 2, 3, 4 })); // 2
Console.WriteLine(Linear.Index(9, new[] { 1, 2, 3, 4 })); // (null)
Console.WriteLine(Linear.Index(2, new[] { 2, 2, 2 }));    // 0
```

```
2
(null)
0
```

El contrato en el caso de duplicados importa. `Index` devuelve la primera ocurrencia — el menor índice. Eso es por convención; algunas bibliotecas devuelven la última. Elige un lado, documéntalo, y nunca dejes que quien llama se pregunte.

La segunda variación es `Count`. En lugar de salir en la primera coincidencia, recorres toda la secuencia y cuentas.

```
public static int Count<T>(T x, IReadOnlyList<T> items)
{
    int c = 0;
    foreach (var y in items)
    {
        if (EqualityComparer<T>.Default.Equals(x, y))
            c++;
    }
    return c;
}
```

Listing 3: `Count`: sin salida temprana

```
Console.WriteLine(Linear.Count(2, new[] { 1, 2, 2, 3, 2 })); // 3
Console.WriteLine(Linear.Count(9, new[] { 1, 2, 3 }));       // 0
Console.WriteLine(Linear.Count('a', "banana"));              // 3 (
    string es IEnumerable<char>)
```

```
3
0
3
```

`Count` no tiene salida temprana — tiene que mirar cada elemento, siempre.

3. Encontrar los extremos

Una pregunta diferente, el mismo paradigma. En lugar de preguntar "¿está x en la secuencia?", preguntas "¿cuál es el elemento más pequeño?" o "¿cuál es el más grande?" El algoritmo tiene la misma forma: recorrerlo todo, mantener el mejor hasta ahora.

"Más pequeño" y "más grande" asumen alguna forma de comparar elementos. En C#, la convención estándar es `Comparison<T>`: un delegado que toma dos `T` y devuelve un entero negativo si el primero es menor, cero si son iguales, positivo si el primero es mayor. Es la misma convención que usa `List<T>.Sort` y `Array.Sort`, y también la de `qsort` en C y `Comparator.compare` en Java.

```
public static int DefaultOrder<T>(T x, T y)
{
    return Comparer<T>.Default.Compare(x, y);
}
```

Listing 4: Orden por defecto

Para el caso común — números, cadenas, cualquier cosa con `IComparable<T>` — `Comparer<T>.Default` ya hace lo correcto, así que `DefaultOrder` solo lo delega.

Ahora `Minimum` se escribe solo. Empieza con el primer elemento como marcador de posición para "el mejor hasta ahora"; recorre el resto; actualiza cada vez que encuentres algo más pequeño.

```
public static T Minimum<T>(IReadOnlyList<T> items, Comparison<T> order =
    null)
{
    order ??= DefaultOrder;
    if (items.Count == 0)
        throw new ArgumentException("La secuencia esta vacia.", nameof(
            items));

    T m = items[0];
    for (int i = 0; i < items.Count; i++)
    {
        if (order(items[i], m) < 0)
            m = items[i];
    }
    return m;
}
```

Listing 5: `Minimum` con comparación parametrizada

La única forma en que `Minimum` puede fallar es si la secuencia está vacía — no hay "el más pequeño de nada". En C# lanzamos `ArgumentException` de forma explícita, que es más idiomático que dejar que un `IndexOutOfRangeException` se propague.

Para `Maximum`, podrías sentir la tentación de escribir una copia casi idéntica con $>$ en lugar de $<$. No lo hagas. La asimetría entre los dos es solo la dirección del ordenamiento — y ya tienes un parámetro que controla la dirección. Invierte sus argumentos.

```
public static T Maximum<T>(IReadOnlyList<T> items, Comparison<T> order =
    null)
```

```
{
    order ??= DefaultOrder;
    return Minimum(items, (x, y) => order(y, x));
}
```

Listing 6: `Maximum` = `Minimum` con el orden invertido

Esa es toda la implementación: `Maximum` es `Minimum` con la comparación invertida. La lambda intercambia los argumentos de `order`, que es la forma más limpia de decir "invierte la dirección de este ordenamiento".

```
var items = new[] { 4, 2, 6, 5, 7, 1, 0 };
Console.WriteLine(Linear.Minimum(items)); // 0
Console.WriteLine(Linear.Maximum(items)); // 7
Console.WriteLine(Linear.Minimum(new[] { "banana", "apple", "cherry" }))
    ; // apple

// Orden personalizado: cadenas por longitud, la mas corta primero
Comparison<string> byLength = (x, y) => x.Length - y.Length;
Console.WriteLine(Linear.Minimum(new[] { "hippopotamus", "cat", "
    elephant" }, byLength)); // cat
```

```
0
7
apple
cat
```

4. Las tres preguntas, aplicadas

Ahora has visto cinco variaciones del mismo algoritmo. La forma es idéntica en cada caso — recorrer cada elemento, hacer algo de contabilidad, devolver. Es hora de plantearle las tres preguntas.

¿Es correcto?

`Find` es correcto porque no hay ningún lugar donde x pueda esconderse. Si x está en la secuencia, entonces hay algún índice i donde `items[i] == x`. El bucle `for` visita cada índice sin excepción. Cuando llega a i, la comprobación de igualdad tiene éxito, y la función devuelve `true`. Si x no está en la secuencia, el bucle termina sin encontrar nunca una coincidencia, y la función devuelve `false`. No hay ningún hueco en el argumento.

Si quieres un sabor más formal, puedes expresar la misma idea como un invariante de bucle:

Invariante de bucle 1. *Al inicio de la iteración i, el objetivo x no ha sido encontrado entre los primeros $i - 1$ elementos.*

El invariante se cumple antes de que empiece el bucle ($i = 0$, ningún elemento examinado, la afirmación es vacuamente verdadera), se preserva en cada iteración (si x coincidió en la iteración $i - 1$, la función habría devuelto `true` y el bucle no estaría ejecutándose), y al terminar te dice que x no está en los primeros n elementos — que es la secuencia entera.

¿Cuán eficiente es?

En el modelo RAM, cada comparación cuesta una unidad de tiempo. El bucle se ejecuta como máximo n veces — exactamente n veces en el peor caso para `Find` (objetivo ausente, o presente en la última posición). Para `Count`, `Minimum` y `Maximum`, el bucle siempre se ejecuta n

veces, ya que no tienen salida temprana. En cualquier caso: tiempo $O(n)$, lineal en el tamaño de la entrada.

El espacio es $O(1)$. La variable del bucle y el "mejor hasta ahora" en ejecución ocupan cada uno memoria constante; la entrada no necesita ser copiada. Nada en el algoritmo crece con n.

¿Es óptimo?

Esta es la interesante. La búsqueda lineal es $O(n)$ — pero ¿es lo mejor posible en $O(n)$? ¿Podría algún algoritmo más astuto responder en, digamos, $O(\sqrt{n})$ o $O(\log n)$ sobre una secuencia no estructurada?

La respuesta es no, y la prueba es el *argumento del adversario*. Imagina que estás jugando un juego contra mí. Traes un algoritmo que afirma decidir si x está en una secuencia de longitud n inspeccionando menos de n elementos — digamos, $n-1$ de ellos. Yo soy el adversario, y elijo qué hay en la secuencia a medida que avanzas — solo tengo que comprometerme con un valor cuando miras una posición.

Inspeccionas $n-1$ posiciones. Hay exactamente una posición que no miraste. Sea cual sea la respuesta que me des — "sí, x está presente" o "no, x no está presente" — puedo falsificarla estableciendo retroactivamente la posición no examinada al valor que te haga equivocarte. Si respondiste que sí sin haber visto nunca x, llenaré la casilla no examinada con otra cosa. Si respondiste que no, llenaré la casilla no examinada con x mismo.

Teorema 1. *Cualquier algoritmo correcto que decida "¿está x en una secuencia no estructurada de longitud n?" debe inspeccionar cada elemento en el peor caso. Por tanto, su complejidad temporal es $\Omega(n)$.*

La búsqueda lineal alcanza esa cota exactamente, lo que la hace *estrechamente óptima* para este problema. Hasta que la entrada tenga más estructura que "una colección ordenada finita donde puedo pedir el elemento i-ésimo", no puedes hacerlo mejor que leerlo todo. No saber nada tiene un precio, y el precio es lineal.

5. El precio de no saber nada

La búsqueda lineal es correcta porque el bucle no tiene ningún lugar donde esconder una coincidencia perdida. Se ejecuta en tiempo $O(n)$ y espacio $O(1)$. Y es óptima — el argumento del adversario dice que ningún algoritmo puede responder a la pregunta "¿está x en esta secuencia?" con menos de n inspecciones en el peor caso, cuando la entrada no tiene estructura.

La búsqueda lineal es la solución a la búsqueda no estructurada — la que cualquier otro enfoque tiene que superar sabiendo algo que el algoritmo de aquí no sabe. El resto del libro es la historia de lo que pasa cuando sabes ese "algo" — cuando la entrada está ordenada, o hasheada, o dispuesta en un árbol, o trazada sobre un grafo.

En el siguiente capítulo conocerás la pieza de estructura más simple que puedes exigir: el orden. Si la secuencia está ordenada, puedes hacerlo dramáticamente mejor que $O(n)$ — y el algoritmo que explota esto es, en mi opinión, el más hermoso del libro.

6. Código completo

```
using System;
using System.Collections.Generic;

namespace Codex.Search
{
    public static class Linear
```

```
    {
        // Encuentra si x esta en items. O(n) tiempo, O(1) espacio.
        public static bool Find<T>(T x, IReadOnlyList<T> items)
        {
            for (int i = 0; i < items.Count; i++)
            {
                if (EqualityComparer<T>.Default.Equals(x, items[i]))
                    return true;
            }
            return false;
        }

        // Menor indice i con items[i] == x, o null si no esta.
        public static int? Index<T>(T x, IReadOnlyList<T> items)
        {
            for (int i = 0; i < items.Count; i++)
            {
                if (EqualityComparer<T>.Default.Equals(x, items[i]))
                    return i;
            }
            return null;
        }

        // Cuenta cuantas veces aparece x. Sin salida temprana.
        public static int Count<T>(T x, IReadOnlyList<T> items)
        {
            int c = 0;
            foreach (var y in items)
            {
                if (EqualityComparer<T>.Default.Equals(x, y))
                    c++;
            }
            return c;
        }

        // Orden por defecto: -1 si x < y, 0 si iguales, 1 si x > y.
        public static int DefaultOrder<T>(T x, T y)
        {
            return Comparer<T>.Default.Compare(x, y);
        }

        // Minimo de una secuencia no vacia, segun el orden dado.
        public static T Minimum<T>(IReadOnlyList<T> items, Comparison<T>
            order = null)
        {
            order ??= DefaultOrder;
            if (items.Count == 0)
                throw new ArgumentException("La secuencia esta vacia.",
                    nameof(items));

            T m = items[0];
            for (int i = 0; i < items.Count; i++)
            {
                if (order(items[i], m) < 0)
                    m = items[i];
            }
            return m;
        }
```

```
63
64          // El maximo es el minimo con la comparacion invertida.
65          public static T Maximum<T>(IReadOnlyList<T> items, Comparison<T>
                order = null)
66          {
67              order ??= DefaultOrder;
68              return Minimum(items, (x, y) => order(y, x));
69          }
70      }
71  }
```

Listing 7: `Linear.cs` — todos los algoritmos

7. Notas y lecturas adicionales

El argumento del adversario es la técnica estándar de cota inferior para búsqueda basada en comparaciones y se trata formalmente en CLRS (4.ª ed.) §8.1 en el contexto del ordenamiento; la misma lógica se aplica aquí. *Algorithms* (4.ª ed.) de Sedgewick y Wayne §1.2 introduce la búsqueda secuencial junto con la API básica. La convención `Comparison<T>` usada aquí (negativo / cero / positivo) es la que usa `List<T>.Sort` y `Array.Sort` de .NET, la que usa `qsort` de C, y la que devuelve `Comparator.compare` de Java; se generaliza de forma más limpia que un booleano *menor que* para los algoritmos basados en comparaciones que vienen más adelante en el libro.